

Formula Neural Networks as Action Policies in Safety-Critical Planning: An Empirical Evaluation

Anonymous submission

Abstract

Formula neural networks (FNNs) are a recently proposed alternative to neural networks (NNs), with Boolean neuron values and hidden-layer nodes defined as propositional formulas over the previous layer. FNNs have been successfully applied to classification and, compared to standard NNs, offer greater interpretability. Here we show that they are also a promising alternative for learning action policies and that they can be more amenable to formal verification. We consider planning under uncertainty, with the objective to reach the goal while avoiding unsafe states. We adapt a recently proposed NN policy safety verification algorithm to FNNs. Our experiments show that, compared to NNs, FNNs can often learn competitive policies while being much faster to verify.

Introduction

Neural networks (NNs) are widely used to represent policies learned through reinforcement learning (RL), and have been highly successful in planning under uncertainty (Mnih et al. 2015; Silver et al. 2016). However, when such policies are used in safety-critical settings, their empirical performance must be complemented by formal safety guarantees. Existing verification methods can provide such guarantees for NN policies, but often at high computational cost.

In this paper, we explore an alternative representation for action policies. The models we consider follow the structure of feed-forward NNs, but their hidden-layer neurons represent logical formulas over binary inputs (Wang et al. 2021; Petersen et al. 2022; Xu, Walter, and Vreeken 2025). As no unified name exists for this family of models, we refer to them as formula neural networks (FNNs). They are trained through differentiable relaxations of logical operations and yield rule-based representations after discretization.

We place our work within a framework recently used in the investigation of policy predicate abstraction (PPA), a policy safety verification algorithm (Vinzent, Sharma, and Hoffmann 2023; Jain et al. 2024). Planning tasks here have initial state as well as action-outcome uncertainty, and use bounded-integer state variables. They are modelled as automata in the JANI language (Budde et al. 2017). The policy’s task is to reach the goal while staying safe, i.e., avoiding states that satisfy a failure formula. Policies are trained via RL, or by imitation learning with a teacher policy; safety is verified posthoc using PPA.

We experiment with different design variants of FNNs. For binarizing the bounded-integer states (which is done at the first FNN layer), we explore two variants: (1) a one-hot encoding of state-variable values, and (2) the conditions appearing in the JANI model. We train the FNNs by imitation learning from NN teachers learned by prior works, using either a classification or a regression loss. We adapt PPA to FNNs in a straightforward manner, by replacing an NN-specialized Satisfiability Modulo Theories (SMT) solver with a generic SMT solver.

Our findings are as follows. First, classification-trained FNNs typically match their neural teachers (sometimes even outperforming them). Regression-trained models also achieve this, but less robustly, with substantially worse performance than their teachers in some domains. Second, PPA is much faster for FNNs than for their neural teachers, often by an order of magnitude or more. Interestingly, regression-trained FNNs tend to be more verifiably safe than classification-trained ones. The configuration yielding the best trade-off between FNN performance vs. safety thus depends on the domain. Third, using conditions from the JANI model instead of one-hot encoding yields considerably smaller FNNs that are more amenable to verification.

Background

We consider planning tasks with probabilistic action outcomes. A task is described by a finite set of state variables $\mathcal{V}$ and a finite set of action labels $\mathcal{A}$. For each variable $v \in \mathcal{V}$, the domain D_v is a non-empty bounded set of integers. A state is a complete assignment over $\mathcal{V}$, and we write $\mathcal{S}$ for the set of all such assignments (states). Each action $a \in \mathcal{A}$ has *guards* specifying when the action is applicable. Guards are boolean combinations of linear constraints over $\mathcal{V}$ of the form $\sum_{i=1}^r d_i v_i + c \bowtie 0$ for some $r \in \mathbb{N}$, $c \in \mathbb{Z}$, coefficients $d_i \in \mathbb{Z}$, variables $v_i \in \mathcal{V}$, and $\bowtie \in \{\leq, =, \geq\}$.

The semantics of the task is a Markov decision process $\Theta = \langle \mathcal{S}, \mathcal{A}, \mathbb{P} \rangle$, where $\mathbb{P}(s, a, s')$ denotes the probability of reaching state s' after applying action a in state s . An action a is applicable in s iff its guard is satisfied in s ; equivalently, iff there exists some $s' \in \mathcal{S}$ such that $\mathbb{P}(s, a, s') > 0$.

A *reach-avoid property* is a tuple $\langle \phi_0, \phi_G, \phi_F \rangle$, where ϕ_0 , ϕ_G , and ϕ_F are linear integer constraints over $\mathcal{V}$ identifying the sets of initial, goal, and failure states, respectively. An *action policy* is a function $\pi : \mathcal{S} \rightarrow \mathcal{A}$. A policy path is a

finite or infinite sequence $\sigma = s_0, a_0, s_1, a_1, \dots$ such that $a_i = \pi(s_i)$ and $\mathbb{P}(s_i, a_i, s_{i+1}) > 0$ for all i . A policy is *unsafe* if there exists a policy path starting in an initial state $s_0 \models \phi_0$ that reaches a failure state before reaching a goal state, i.e., there exists some $n \geq 0$ such that $s_n \models \phi_F$ and $s_i \not\models \phi_G$ for all $0 \leq i < n$. Otherwise, the policy is *safe*. Note that *safety does not imply goal reaching*: a safe policy may also avoid both ϕ_G and ϕ_F by cycling.

Formula Neural Networks. We model action policies using the FNN architecture proposed by Wang et al. (2021, 2024); hereafter, FNN refers to this model, not to the general concept. As illustrated in Figure 1, FNNs follow the overall structure of standard neural networks, but hidden neurons represent propositional formulas rather than continuous activations. An FNN consists of a binarization layer, $L > 0$ logical layers, and a final linear output layer.

The binarization layer produces binary inputs $u^{(0)} \in \mathbb{B}^{n_0}$, where $\mathbb{B} = \{0, 1\}$ denotes the Boolean domain and $n_0 \in \mathbb{N}$ is the input dimension. Each logical layer maps a Boolean vector from the previous layer to another Boolean vector. Each neuron is either an AND (conjunction) or an OR (disjunction) neuron. A neuron i in layer ℓ selects a subset of inputs from the previous layer via a binary weight vector $W_i^{(\ell)} \in [0, 1]^{n_{\ell-1}}$. Let $\text{pred}(i) = \{j \mid W_{i,j}^{(\ell)} \geq 0.5\}$ denote the predecessors of neuron i . The neuron then computes

$$u_i^{(\ell)} = \begin{cases} \bigwedge_{j \in \text{pred}(i)} u_j^{(\ell-1)} & \text{if } i \text{ is an AND neuron,} \\ \bigvee_{j \in \text{pred}(i)} u_j^{(\ell-1)} & \text{if } i \text{ is an OR neuron.} \end{cases}$$

So each neuron represents a propositional subformula over the binarized inputs $u^{(0)}$. The neurons $u_i^{(L)}$ in the last (logical) layer define full formulas ϕ_i over $u^{(0)}$, which are combined by a linear layer to produce a real-valued output vector. FNNs can be trained with gradient descent using surrogate gradients for the discrete logical operations; we refer to Wang et al. (2024) for details on the specific method.

Safety Verification. Policy predicate abstraction (PPA) has recently been proposed as an algorithm to verify whether a given learned policy is safe (Vinzent, Sharma, and Hoffmann 2023). PPA builds an overapproximating abstraction defined through a set of predicates (linear constraints over state variables). Crucially, this overapproximation pertains to the state-space subgraph restricted to policy-induced transitions. Hence, if there does not exist a path from ϕ_0 to ϕ_F in the abstraction, then the policy is safe. Otherwise, the set of predicates is refined using CEGAR (Clarke et al. 2003) until the policy is proved safe or unsafe. The key computational problem in PPA is deciding which transitions are valid in the abstraction, this is formulated as an SMT query. Each query combines state-transition requirements defined by the planning task with an encoding of the policy function π to determine whether a policy-induced transition between states exists in the abstraction. Vinzent, Sharma, and Hoffmann (2023) use *Marabou* (Katz et al. 2019), an SMT solver specialized for NNs, to process these queries. PPA is readily applicable to other learned policy representations, provided that a solver exists for the abstract transition queries.

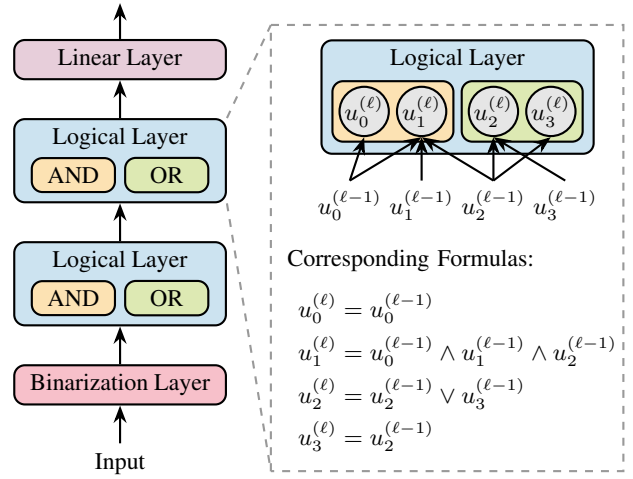


Figure 1: *FNN architecture with two hidden layers*, adapted from (Wang et al. 2021). Inputs are binarized and processed by logical hidden layers whose neurons learn propositional formulas over binary (Boolean) inputs $u^{(\ell-1)}$. In our setting, half of the neurons compute conjunctions and half compute disjunctions, followed by a standard linear output layer.

FNN Policies

We construct FNN policies as follows. Given a state $s \in \mathcal{S}$, we define the binarization layer β that produces a Boolean feature vector $u^{(0)} := \beta(s) \in \mathbb{B}^{n_0}$. In our setting, β is instantiated either by *one-hot features* or by *JANI features*. The former is the standard encoding of variable-value pairs, where each pair is represented by a distinct boolean feature. The latter is derived from the guards and reach-avoid constraints. More specifically, for each action guard and for the formulas defining ϕ_G and ϕ_F , we decompose conjunctions and disjunctions into subformulas and use the resulting set of constraints as features, such as $x = 3$, $y \leq 5$, or $x = y + 1$.

The neurons in the last logical layer of the FNN can thus be interpreted as propositional formulas $\phi_1, \dots, \phi_{n_L}$ over the state, whose truth values correspond to the outputs of that layer $\phi_i(s) := u_i^{(L)}(s)$.

The linear layer then assigns each action $a \in \mathcal{A}$ a real-valued score given by a weighted sum of the formulas represented in the last logical layer:

$$\text{score}_a(s) = b_a + \sum_{i=1}^{n_L} W_{a,i} \phi_i(s),$$

where $W_{a,i} \in \mathbb{R}$ is the weight of formula $\phi_i \in \mathbb{B}$ for action a , and $b_a \in \mathbb{R}$ is a bias term. Learned policies may select any action in any state, even if it is not applicable. An established technique to avoid this is to filter the inapplicable actions prior to selecting an action (Toyer et al. 2020; Ståhlberg, Bonet, and Geffner 2022). Hence, the FNN policy decision is defined as the action with the highest score among the applicable actions. $\text{app}(s)$ in state s :

$$\pi_{fnn}(s) := \arg \max_{a \in \text{app}(s)} \text{score}_a(s).$$

We train FNN policies with imitation learning using NN policies as teachers. For each problem, we sample traces from the teacher and collect states together with the teacher’s output. We consider two types of FNN policies: *classifiers* trained to predict the teacher-selected action by minimizing the cross-entropy loss, and *regressors* trained to predict the full vector of actions values returned by the teacher by minimizing the mean squared error. The training dataset is shuffled and split into five folds. In each run, one fold is used for testing, and 20% of the remaining data are held out for validation while the rest are used for training.

For post-hoc safety verification of FNN policies, we use PPA by Vinzent, Sharma, and Hoffmann (2023), starting from the authors’ C++ codebase. To adapt PPA from NN verification to FNN verification, we only need to replace the policy representation in the abstract transition queries. To this end, we use a straightforward encoding of FNNs into SMT, and call Z3 (de Moura and Bjorner 2008) to solve the resulting queries in PPA.

Experiments

To train FNNs for our setting we adapted the codebase of Wang et al. (2024). The code for all experiments will be made available on acceptance. We ran experiments comparing (1) the policy quality and (2) the verifiability of FNNs to their neural teachers. In what follows, we describe our experiment setup and discuss the main results. Training was performed on NVIDIA A100 and P100 GPU nodes. Evaluation and verification experiments were conducted on machines with AMD EPYC 9654 processors at 2.4GHz with time and memory limits of 24 hours and 16GB respectively.

Benchmarks We use 26 benchmark instances from Jain et al. (2025), each defined by a JANI model and a trained NN policy on that model. These benchmarks have been modified by prior work, with initial-state and action-outcome non-determinism added. We focus on discrete domains with bounded integer-valued variables, so we rely in a straightforward one-hot encoding of state-variable values. The JANI models are probabilistic versions of the planning benchmarks *Blocksworld* and *Transport*; an abstract version of a logistics problem at Airbus *Beluga*; as well as variants of a transportation domain, *one way line (1WL)* and *two way line (2WL)*, modelling a truck moving along a line of locations with a chance of slipping on icy regions. The neural policies are feedforward NNs, trained with Q-learning to maximize the expected reward of reaching the goal. To obtain good teachers on benchmarks where the neural policy performs poorly, we trained new NN policies using Q-learning and selected the ones with the highest average reward on sample runs from 10,000 sampled start states.

Experiment Setup To train the FNNs, we fix the number of logical layers to 1 and vary hyperparameter settings for the binarization layer (one-hot and JANI features), the learning rate $\{0.01, 0.005\}$ and the width of the logical layer $\{128, 256, 512\}$. To choose amongst the trained policies, we evaluate them through simulation runs from a fixed set of 50,000 sampled start states. We consider two metrics: (1)

%Goal the percentage of runs in which the policy successfully reaches a goal state and (2) **%Fail** the percentage of runs where the policy reaches a failure state. Note that neither outcome is guaranteed as a policy may cycle indefinitely without reaching either.

Model Selection In the targeted application scenario, it is quite practical for the user to make hyperparameter choices on a per-instance basis. After all, the policy is designed to be used for many runs on a given instance (exponentially many possible initial states and action-outcome combinations). For performance comparisons, we select FNNs by %Goal, then %Fail, then width. For verification-focused results, we select by Safe > Unsafe > TO, then %Goal, %Fail, with ties broken by smaller logical-layer width. Selection is per benchmark/objective, and per binarization types when they are reported separately.

Results

Here we discuss the main results of our experiments. We focus on three questions: (1) Can FNNs match the performance of their neural teachers? (2) Are FNNs easier to verify compared to their neural teachers? (3) Does the binarization layer make a difference?

Q1. Can FNNs match the performance of their neural teachers? Figure 2 shows the FNN performance relative to NN teachers (on 18 of 26 benchmarks for space reasons; the main observations remain the same). For classification-trained FNNs (left-hand side of Figure 2), the answer is yes. The best FNNs offer similar %Goal and %Fail as their neural teachers. They are rarely worse, and sometimes substantially better (e.g. on benchmark 2, 1WL Det (40, 25), where the FNN has 29% more sample runs reaching the goal). For regression-trained FNNs (right-hand side), the results are more limited. While regression preserves NN-like performance on some benchmarks, it often reduces goal-reaching performance.

Q2. Are FNNs easier to verify compared to their neural teachers? The answer is yes as shown in Figure 3. Verification of the neural teachers was feasible on only 5 of 26 benchmarks within the 24-hour time limit. In contrast, verification for classification-trained and regression-trained FNNs completed in 12 and 16 instances respectively, often an order of magnitude or more faster than NN verification (points clearly below the line in the log-log plot).

Most of the FNN verification outcomes are unsafe, in particular for classification-trained FNNs. For regression-trained FNNs, there are 4 proved-safe cases, like for NNs. In particular, the regression-trained FNN for *Transport* is verified safe despite the neural teacher itself being unsafe. So regression training seems better at finding safe policies. The runtime advantage of PPA for FNN vs. NN policies is also large: the largest safe-case improvements are from 1523s to 35s and from 22528s to 43s.

One may wonder whether the NN teachers of the unsafe FNN policies are safe. In most cases, this is not so: in 18 of the 21 instances on which the neural teacher verification

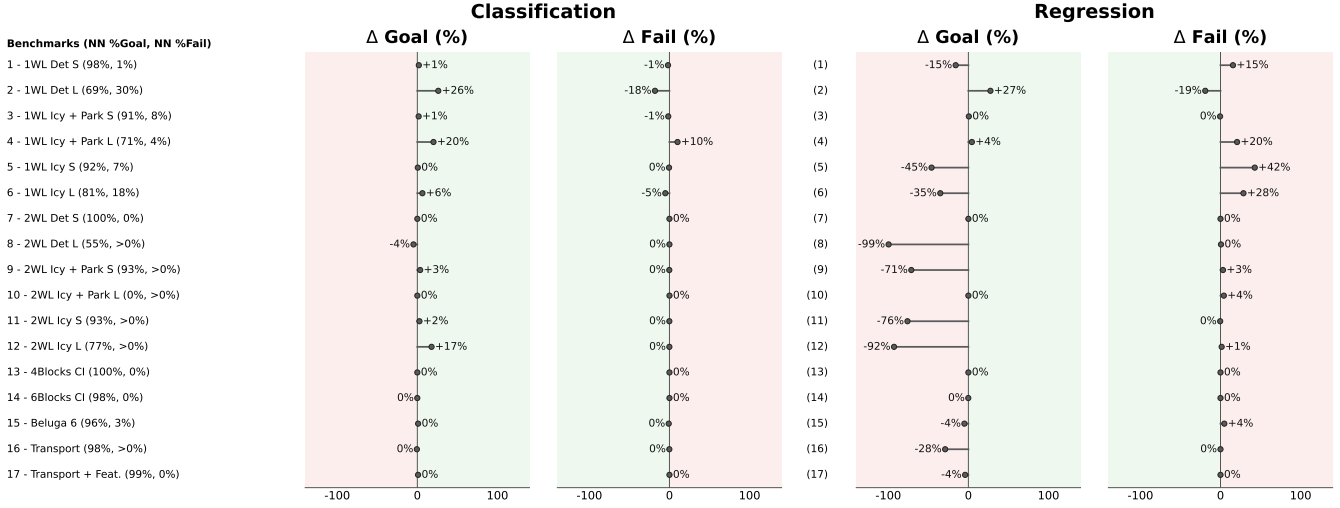


Figure 2: *FNN policy quality relative to NN teachers.* FNNs use the performance criterion above. Labels show teacher (%Goal, %Fail); $> 0\%$ means positive but below 1%. We show ΔGoal , i.e., $\%Goal_{\text{student}} - \%Goal_{\text{teacher}}$, and ΔFail , i.e., $\%Fail_{\text{student}} - \%Fail_{\text{teacher}}$, with exact deviations annotated. Positive ΔGoal and negative ΔFail are desirable.

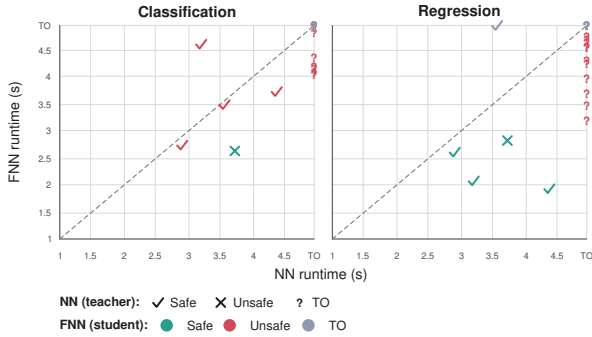


Figure 3: *Verification runtimes for FNN policies compared to their neural teachers.* FNNs use the verification criterion above. Axes show runtime in seconds (log10); TO is the 24-hour timeout. Shape marks the NN outcome, while color marks whether the selected FNN is proved safe, unsafe, or times out.

timed out, we found unsafe policy runs by sampling. However, sampling can only reveal unsafe runs; it may miss violations and cannot prove their absence. Thus, even proving unsafety with PPA is important, as it provides a certificate that sampling alone may fail to find.

Q3. Does the binarization layer make a difference? As shown in Table 1, the answer is yes: JANI features reduce average rule size by around a factor of 5. This directly benefits PPA because SMT query size is linked to FNN size, and large rules yield large conjunctions and disjunctions. Consequently, the smaller rules reduce timeouts by 5 for classification-trained FNNs and by 11 for regression-trained FNNs.

Model	Obj.	$ \phi $	Evaluation		Verification		
			%Goal	%Fail	✓	✗	TO
NN	—	—	83	6	4	1	21
FNN (OH)	Class.	47	81	6	0	7	19
FNN (P)	Class.	6	82	5	1	11	14
FNN (OH)	Reg.	65	52	18	2	3	21
FNN (P)	Reg.	8	57	12	4	12	10

Table 1: *Effect of binarization.* FNN rows aggregate verification-selected models. We report training objective, average literals per rule, policy quality (%Goal, %Fail), and verification outcomes: safe (✓), unsafe (✗), or timeout (TO).

Conclusion

Neural policies are widely used for planning problems, but their verification remains challenging. Our results show that FNNs can serve as action policies while offering a more verification-friendly representation. In particular, using JANI predicates as binarization features yields much smaller rules, improves verifiability, and can preserve or improve policy quality.

Regression-based imitation learning of the FNN is better at learning to be safe as it gets a training signal on the magnitude of differences between actions, in particular negative values on unsafe actions. The downside is that, the capability of FNN to faithfully model the numeric outcomes of an NN is limited. In some domains, this leads to worse goal-reaching performance.

More broadly, our results open the path to deeper investigation of FNNs in planning and motivate the search for alternative policy representations aiming verifiability.

Acknowledgments

This work was funded by DFG Grant 389792660 as part of TRR 248 (CPEC, <https://perspicuous-computing.science>).

References

- Budde, C. E.; Dehnert, C.; Hahn, E. M.; Hartmanns, A.; Junges, S.; and Turrini, A. 2017. JANI: Quantitative Model and Tool Interaction. In *Proceedings of the 23rd International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'17)*, 151–168.
- Clarke, E.; Grumberg, O.; Jha, S.; Lu, Y.; and Veith, H. 2003. Counterexample-guided abstraction refinement for symbolic model checking. *Journal of the Association for Computing Machinery*, 50(5): 752–794.
- de Moura, L.; and Bjorner, N. 2008. Z3: An Efficient SMT Solver. In Ramakrishnan, C.; and Rehof, J., eds., *Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2008*, LNCS 4963. Berlin, Heidelberg: Springer. https://doi.org/10.1007/978-3-540-78800-3_24.
- Jain, C.; Cascioli, L.; Devos, L.; Vinzent, M.; Steinmetz, M.; Davis, J.; and Hoffmann, J. 2024. Safety Verification of Tree-Ensemble Policies via Predicate Abstraction. In *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI'24)*.
- Jain, C.; Sherbakov, D.; Vinzent, M.; Steinmetz, M.; Davis, J.; and Hoffmann, J. 2025. Policy Safety Testing in Non-Deterministic Planning: Fuzzing, Test Oracles, Fault Analysis. In *Proceedings of the 28th European Conference on Artificial Intelligence (ECAI'25)*.
- Katz, G.; Huang, D. A.; Ibeling, D.; Julian, K.; Lazarus, C.; Lim, R.; Shah, P.; Thakoor, S.; Wu, H.; Zeljic, A.; Dill, D. L.; Kochenderfer, M.; and Barrett, C. 2019. The Marabou Framework for Verification and Analysis of Deep Neural Networks. In Dillig, I.; and Tasiran, S., eds., *Computer Aided Verification. CAV 2019*, LNCS 11561. Cham: Springer. https://doi.org/10.1007/978-3-030-25540-4_26.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M. A.; Fidjeland, A.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.
- Petersen, F.; Borgelt, C.; Kuehne, H.; and Deussen, O. 2022. Deep differentiable logic gate networks. *Advances in Neural Information Processing Systems*, 35: 2006–2018.
- Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; Dieleman, S.; Grewe, D.; Nham, J.; Kalchbrenner, N.; Sutskever, I.; Lillicrap, T.; Leach, M.; Kavukcuoglu, K.; Graepel, T.; and Hassabis, D. 2016. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature*, 529: 484–503.
- Ståhlberg, S.; Bonet, B.; and Geffner, H. 2022. Learning Generalized Policies without Supervision Using GNNs. In *Proceedings of the 19th International Conference on Principles of Knowledge Representation and Reasoning (KR'22)*.
- Toyer, S.; Thiébaux, S.; Trevizan, F. W.; and Xie, L. 2020. ASNNets: Deep Learning for Generalised Planning. *Journal of Artificial Intelligence Research*, 68: 1–68.
- Vinzent, M.; Sharma, S.; and Hoffmann, J. 2023. Neural Policy Safety Verification via Predicate Abstraction: CEGAR. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI'23)*, 15188–15196. AAAI Press.
- Wang, Z.; Zhang, W.; Liu, N.; and Wang, J. 2021. Scalable Rule-Based Representation Learning for Interpretable Classification. In Ranzato, M.; Beygelzimer, A.; Dauphin, Y. N.; Liang, P.; and Vaughan, J. W., eds., *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, 30479–30491.
- Wang, Z.; Zhang, W.; Liu, N.; and Wang, J. 2024. Learning Interpretable Rules for Scalable Data Representation and Classification. *IEEE Trans. Pattern Anal. Mach. Intell.*, 46(2): 1121–1133.
- Xu, S.; Walter, N. P.; and Vreeken, J. 2025. Neural Rule Lists: Learning Discretizations, Rules, and Order in One Go. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.